

Project Blueprint: Employee Knowledge Assistant

Objective: Build a secure Retrieval-Augmented Generation (RAG) system to answer internal employee queries using fictional corporate documentation.

1. Document Ingestion Scope

The system index covers the following corporate knowledge baselines:

- **Employee & HR Handbook:** Code of conduct, onboarding procedures, workplace rules, and general organizational structure.
- **Leave & Travel Policy:** Guidelines for annual leave, parental leave, sick leave tracking, and corporate travel expense limits.
- **IT & Cybersecurity Policies:** Asset usage conditions, password rotation rules, remote work security parameters, and access provisioning.
- **Benefits & FAQs:** Medical insurance coverage metrics, retirement matching schedules, and standard corporate FAQs.

2. Technical Architecture Blueprint

The technical stack leverages a standardized RAG architecture to guarantee low latency and precise document grounding:

Component	Technology Recommendation	Function
Data Ingestion	PyPDF / Unstructured	Parses PDFs, Markdown files, and text docs.
Chunking & Vectorization	LangChain / TextSplitter	Splits text recursively with chunk overlap.
Embedding Model	HuggingFace (e.g., BGE-large) / OpenAI	Converts chunks into high-dimensional vectors.
Vector Database	Chroma / FAISS / Pinecone	Indexes embeddings for fast semantic lookup.
LLM Reasoning Engine	Llama-3 (Local) / GPT-4o	Generates answers strictly grounded in context.
User UI Interface	Streamlit / Chainlit	Provides clean conversational chat layout.

3. Step-by-Step Implementation Flow

- Step 1: Document Preparation:** Collate all policies into a structural data repository (e.g., data/). Ensure text layouts are clean.
- Step 2: Processing Pipeline:** Implement standard recursive chunking (size: 512 tokens, overlap: 64 tokens) to preserve contextual boundaries.

Step 3: Vector Storage: Initialize your vector DB instance and pass the chunked data array through your embedding wrapper to store vectors locally.

Step 4: RAG Prompt Configuration: Formulate an explicit system prompt: *'Answer the query using ONLY the provided text snippets. If the text does not contain the answer, reply that you do not know.'*

Step 5: Deployment: Spin up a Streamlit user interface featuring text input fields, conversation history buffers, and citations mapping back to specific source policies.